Obsidian — Production Tech Stack (React + Vite Edition)

Full Frontend + Backend Technology Blueprint

Architecture Type:

Real-Time Virtual Office Collaboration Platform

Optimized For:

- Indian startups
- Low infrastructure cost
- Discord-style voice collaboration
- High scalability
- Real-time team communication

Reference Architecture:

---

1. Core Architecture Philosophy

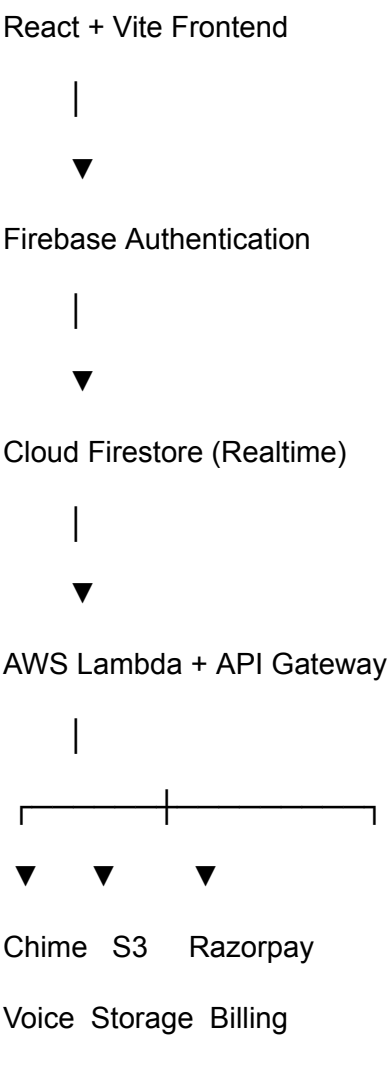
The entire architecture is designed around:

Goal	Strategy
Fast UI Performance	React + Vite
Low Backend Costs	Firebase + AWS Serverless
Real-Time Communication	Firestore + Chime SDK

Massive Scalability	Serverless Infrastructure
Fast Development	TypeScript-first architecture

---

2. Complete System Architecture



---

3. Frontend Tech Stack (React + Vite)

3.1 Core Frontend Stack

Technology	Purpose	Why Chosen
React 19	UI framework	Ecosystem maturity
Vite	Build tool	Extremely fast development
TypeScript	Type safety	Scalable architecture
React Router DOM	Client routing	SPA navigation
SWC	Fast compilation	Better dev performance

3.2 Styling & UI Stack

Technology	Purpose
Tailwind CSS	Utility-first styling
shadcn/ui	Component system

Framer Motion	UI animations
Lucide React	Icons
clsx	Conditional styling
tailwind-merge	Tailwind conflict handling

3.3 State Management Stack

Technology	Purpose
Zustand	Global state
TanStack Query	API/server caching
React Context API	Lightweight contexts

3.4 Authentication Stack

Technology	Purpose

Firebase Authentication	Login/signup
Google OAuth	Social login
GitHub OAuth	Developer login
JWT Tokens	Session validation

3.5 Real-Time Infrastructure

Technology	Purpose
Firestore SDK	Realtime messaging
Firebase Presence	User online status
Snapshot Listeners	Live updates
WebSocket Fallback	Advanced event sync

3.6 Voice Communication Stack

Technology	Purpose
Amazon Chime SDK JS	Voice rooms
WebRTC	Media transport
AudioWorklets	Audio processing
MediaDevices API	Mic/speaker control

3.7 Forms & Validation Stack

Technology	Purpose
React Hook Form	Form management
Zod	Validation
Hookform Resolvers	Zod integration

3.8 Frontend Performance Stack

Technology	Purpose
React.lazy	Code splitting
Suspense	Lazy rendering
Vite HMR	Instant hot reload
Virtualized Lists	Large message rendering

3.9 Frontend Monitoring Stack

Technology	Purpose
Sentry	Error tracking
Firebase Analytics	User analytics
LogRocket	Session replay
PostHog	Product insights

3.10 Frontend Deployment Stack

Technology	Purpose
Vercel	Frontend hosting
Netlify (alternative)	Static deployment
Cloudflare CDN	Global caching
GitHub Actions	CI/CD

---

4. Backend Tech Stack

4.1 Core Backend Infrastructure

Technology	Purpose
AWS Lambda	Serverless backend
API Gateway	API routing

Node.js 22	Runtime
TypeScript	Backend maintainability

4.2 Authentication & Identity

Technology	Purpose
Firebase Auth	User management
Firebase Admin SDK	Token validation
JWT	Session security

4.3 Database Stack

Technology	Purpose
Cloud Firestore	Primary realtime database
Firestore Rules	Authorization

Firebase Functions	Trigger automation

4.4 Voice Infrastructure Stack

Technology	Purpose
Amazon Chime SDK	Voice meetings
Chime Meeting APIs	Session generation
Chime Media Pipelines	Future recording

4.5 File Storage Stack

Technology	Purpose
AWS S3	File storage
Signed URLs	Secure uploads
CloudFront	CDN delivery

S3 Lifecycle Rules	Cost optimization

4.6 Billing & Subscription Stack

Technology	Purpose
Razorpay API	Payments
Razorpay Webhooks	Subscription updates
AWS Secrets Manager	Secret management

4.7 Backend Security Stack

Technology	Purpose
AWS WAF	API protection
IAM Policies	AWS access control
HTTPS TLS 1.3	Secure transport

HMAC Validation	Razorpay verification

4.8 Event & Queue Processing

Technology	Purpose
AWS EventBridge	Event orchestration
AWS SQS	Queue management
Lambda Triggers	Async jobs

4.9 Backend Monitoring Stack

Technology	Purpose
AWS CloudWatch	Logs & metrics
AWS X-Ray	API tracing
Sentry	Backend monitoring

---

5. Frontend Folder Structure

src/

│

├── assets/

├── components/

│ ├── ui/

│ ├── chat/

│ ├── voice/

│ ├── workspace/

│ └── billing/

│

├── pages/

│ ├── auth/

│ ├── dashboard/

│ ├── settings/

│ └── workspace/

│

├── hooks/

├── services/

├── stores/

├── contexts/

├── layouts/

├── routes/

├── utils/

├── lib/

├── types/

├── styles/

└── main.tsx

---

6. Backend Folder Structure

backend/

│

├── functions/

├── routes/

├── middleware/

├── services/

├── repositories/

├── controllers/

├── schemas/

├── utils/

├── events/

├── config/

└── types/

---

7. DevOps Stack

7.1 CI/CD Pipeline

GitHub Push

↓

GitHub Actions

↓

Lint + Test

↓

Build Vite App

↓

Deploy Frontend

↓

Deploy AWS Lambda

7.2 Infrastructure Tools

Technology	Purpose
GitHub Actions	CI/CD

Docker	Local development
Terraform	Infrastructure as Code
Serverless Framework	Lambda deployment

---

8. Testing Stack

8.1 Frontend Testing

Technology	Purpose
Vitest	Unit testing
React Testing Library	Component testing
Playwright	E2E testing
Storybook	UI development

8.2 Backend Testing

Technology	Purpose
Jest	Backend tests
Supertest	API testing
Firebase Emulator	Integration tests

---

9. Security Architecture

9.1 Frontend Security

Technology	Purpose
DOMPurify	XSS sanitization
CSP Headers	Browser protection
Secure Cookies	Token protection

9.2 Backend Security

Technology	Purpose
Firebase Admin SDK	Auth verification
AWS IAM	Resource permissions
AWS WAF	DDoS protection
Rate Limiting	Abuse prevention

---

10. Voice System Tech Stack

10.1 Voice Join Flow

React Client

↓

Voice Join Request

↓

AWS Lambda Validation

↓

Amazon Chime Meeting

↓

Attendee Token Generated

↓

React Connects To Voice Room

---

11. Database Architecture

11.1 Firestore Collections

workspaces/

users/

channels/

messages/

roles/

subscriptions/

notifications/

voiceSessions/

files/

activityLogs/

11.2 Firestore Optimization

Strategy	Purpose
Composite Indexes	Faster queries
Batched Writes	Lower costs
Pagination Cursors	Infinite scrolling
Offline Cache	Better UX

---

12. Deployment Architecture

Frontend Deployment

Service	Purpose
Vercel	Main frontend hosting
Cloudflare	CDN + security

Backend Deployment

Service	Purpose
AWS Lambda	Backend compute
API Gateway	API routing
Firestore	Database
S3	Storage

---

13. Production Performance Targets

Metric	Target
Initial Load	< 2 seconds
Message Delivery	< 300ms
Voice Join Time	< 1 second
API Response	< 500ms

---

14. Cost Optimization Strategy

Frontend

Strategy	Benefit
Vite Bundling	Smaller builds
Lazy Loading	Reduced payload
CDN Caching	Faster delivery

Backend

Strategy	Benefit
Lambda	Pay-per-use
Firestore	No fixed DB servers
S3 Lifecycle	Reduced storage costs

---

15. Scalability Targets

Component	Scale
Concurrent Users	50,000+
Active Voice Users	10,000+
Messages/Day	10 Million+
Workspaces	100,000+

---

16. Future Expansion Stack

Phase 2

Technology	Purpose
OpenAI API	AI summaries
Whisper API	Voice transcription
LangChain	AI orchestration

Phase 3

Technology	Purpose
Electron	Desktop app
React Native	Mobile app
Meilisearch	Advanced search

---

17. Final Recommended Production Stack

Frontend

React 19

Vite

TypeScript

Tailwind CSS

shadcn/ui

Zustand

TanStack Query

Firebase SDK

Amazon Chime SDK

Backend

AWS Lambda

API Gateway

Node.js

TypeScript

Firebase Admin SDK

Cloud Firestore

Amazon Chime SDK

AWS S3

Razorpay

---

18. Final Technical Summary

This React + Vite architecture provides:

- Extremely fast frontend performance
- Low development complexity
- Real-time collaboration infrastructure
- Production-grade voice systems
- Serverless scalability
- Low operational cost

The architecture is optimized specifically for:

- Startup MVP speed
- Indian market affordability
- Massive future scalability
- Discord-like user experience

- Lean engineering teams